1

If G is a subset of a k-tree then G has tree width k

Proof by induction

base case is G is subgraph of K_{k+1}, our tree width k is from putting the $k+1$ vertices in one bag so any G subgraph of K_{k+1} would have this tree width k

Inductive step, assume works for $G-v$, show works for G, specifically v is the last vertex added to make G.

As demonstrated in the base case, we really only care about showing the if this k-tree has tree width k because if we show a graph has tree width k then all subgraphs of it have tree width k (because we can use the same tree decomposition for both the k-tree and the subgraphs). Thus we are trying to show that if $G-v$ is a k tree with tree width k then G also has tree width k (assuming G is also a k-tree)

By the definition of k-tree, we know that the neighbors of v form a k clique and that by definition of tree decompositions we know that every edge in G is in our T. Thus the only new edges we are adding can fit in a bag of $\{v, N(v)\}$ and add this bag somewhere to the tree decomposition of $G-v$ to form a tree decomposition of G

Well if there exists a bag in the tree decomposition of $G-v$ with all the neighbors of v and another vertex, then we can add on our new bag to this vertex and this would be a valid tree decomposition of G. We can also show that by the definition of a k-tree that we can force such a tree decomposition. From the base case we see that the bag is a K_{k+1} clique and that from definition of k-tree adding vertices adds a vertex to an existing k clique. In other words in the tree decomposition we construct our tree in the order the vertices were added with the bags always containing every possible k clique allowing us to add a bag for each new vertex in our k-tree construction.

2

Idea + Fundamental Lemmas / Properties

We start with the hint idea with finding all vertex cuts of size $k+1$ in H. Note that if no such cut exists then H is not isomorphic to G because of the proofs we did in class on monday which showed that $\kappa(G) \leq k$ for G having tree width k, and also if theres a vertex mismatch or edge mismatch again easy return false. This part works with doing $\binom{n}{k+1} \leq n^{k+1}$ where n is the number of vertices. So we pick $k+1$ vertices of H, and run BFS on an arbitrary root vertex and see if it able to disconnect the graph (we didnt reach all $n-k-1$ vertices). This allows us to partition $

From class, we discussed how each bag in T_G is able to separate the graph (except leaf nodes) so our H bags should also do the same. We also showed from Nate's proof that we are able to modify T_G to have each bag be filled with $k+1$ vertices. We also know that we can modify the tree such that adjacent bags have some vertices in common (also shown in class on wednesday).

Thus the general idea for the algorithm is find possible projections of bags in T_G onto vertices of H (constrained to valid edges in G) then build up and join making sure the projections from the bags are valid.

Actual Algorithm

Find all $\binom{n}{k+1}$ sets of vertices in H and split into the ones that disconnect H and do not disconnect H. This is less than n^{k+1} so we will approximate this as $O(n^{k+1})$. Checking for disconnection is $O(|G|)$ so this is still polynomial time.

Now for some leaf bag B_i of T_G we look at all the $k+1$ sets of H that do not disconnect the graph. For each $k+1$ set, go through all possible projections of the $k+1$ vertices in B_i to the $k+1$ vertoces in H and check if it is a valid isomorphism while adhering to the vertices of G. We can call each check as $O(|G|)$ but it would be much lower than that simply because we are looking at $k+1$ vertices. If it is a valid isomorphic projection we store True in our table otherwise false. Now doing this for all leaf node bags we have each bag as $O(n^{k+1}(k+1)!)$ storage and computed in $O(n^{k+1}(k+1)!|G|)$ runtime which is polynomial because k is a constant and then our runtime is $\sim O(|G|^{k+2})$ per each leaf node bag. So to be explicit, each Bag has the following $O(n^{k+1}(k+1)!)$ table:

Component	Cardinality of Component
$k+1$ nodes in $S \subset H$	$O(n^{k+1})$
Projection f for $v \in B, u \in S$ s.t $f(v) \to u \wedge v = f^{-1}(u)$	$O((k+1)!)$
Binary is isomorphic	2 values but one per entry

Now for the next layer of the tree (were traversing like a backwards BFS, from leaves to a root) for a specific Bag we iterate over the $O(n^{k+1)}$ $k+1$ sets of H that disconnect H. Now we do something slightly similar to what was done from wednesday with the quick note that we only case about adjacent bags to a specific Bag because all the issues that would arise from a (great^d) grandchild was already handled by the child bag. Now we need a way to cleanly handle the work from the children.

So first we compute the same table in the same way as the leaf nodes but for this new Bag. Now for each entry in the table that is labelled as an isomorphic subgraph then we iterate through the child bags B_C (which have their tables filled out) and specifically check that for $v \in B \cap B_C$ that $f_B(v) = f_{Bc}(v)$ and for $v \notin B \cap B_C$ that $f_B(v) \neq f_{Bc}(v)$ and that if there is an isomorphic

transformation in B_C (ie one row is true) then the row in B stays true otherwise turn it to false. We do this for all child Bags so this would be runtime $O(n^{k+1}(k+1)! \times (|B|n^{k+1}(k+1)!))$ and we showed in class on wednesday that $|B|$ is $O(|G|)$ so this is still polynomial runtime and same space because we are editing the existing $O(n^{k+1}(k+1)!)$ table.

Thus our total runtime is something like (doing substituting n for $|G|$ so the notation is slightly clearer) $O(n(n^{2k+3}(k+1)!)) = O(n^{2k+4}(k+1)!)$